DAILY FUSION CODE · WEEK 2

How small is small? Machine epsilon

Python | Mon 19 Oct 2026 | NUM-1.1 | Level 1 | ~15 min

A computer stores a number like 1.0 or 10^{20} with only about 16 significant digits. Today you measure that limit yourself: the **machine epsilon** ε, the smallest relative change a 64-bit float can still see. Then you check what it means for a plasma with 10^{20} ions. Every error estimate later in the NUM track rests on this number.

New to floating-point numbers? Read the theory first.

Syntax you'll need

```
x = 1.0
while x > 0.1:            # repeat as long as the condition is True
    x = x / 2             # 1.0 -> 0.5 -> 0.25 -> ...
count = count + 1         # add one to a counter
a == b                    # True or False
print(f"{x:.6e}")         # scientific notation, 6 decimals
```

Tasks

1. **Find ε by halving** in `machine_epsilon()`. TODO 1a: the `while` condition, "$1.0 + \varepsilon/2$ is still bigger than 1.0". TODO 1b: halve `eps`.

   Hint: the loop stops at the last `eps` for which adding it to 1.0 still changes the result. Compare your value with NumPy's line printed under it.

2. **Adding one ion to** 10^{20} in `main()`. TODO 2a: estimate the gap between $N = 10^{20}$ and the next float as $N\varepsilon$. TODO 2b: test whether `N + added` is still equal to `N`. TODO 2c: complete the sentence at the bottom of the file.

   Hint: for 2b, `==` already gives `True` or `False`, so `same = (N + added == N)` needs no `if`.

Check yourself

You should get $\varepsilon \approx 2.22 \times 10^{-16}$, and `N + 1 == N` prints `True`.

Submit

Commit and push `solution.py`. Optional: replace `??` on its first line with how long it took you (e.g. `# time: 15 min`).

If you have time: try `1e308 * 10` and `np.exp(-800.0)` and look up both results in the theory note.

THEORY

Floating-point numbers and machine epsilon

The foundation under every NUM topic. Tao Pang's chapter 1 gives the background but is brief on number storage, so this note stands on its own.

How a float is stored

A 64-bit float ("double", Python's `float`) is scientific notation in base 2:

$$x = \pm 1.b_1 b_2 \ldots b_{52} \times 2^{e}$$

There are **52 bits** after the binary point (the mantissa) and an exponent e from about -1022 to 1023. So every float has the same *relative* precision, about 16 decimal digits, whether it is 10^{-10} or 10^{20}.

Machine epsilon

The **machine epsilon** ε is the gap between 1.0 and the next float above it. Anything much smaller than ε added to 1.0 is rounded away. With m mantissa bits, $\varepsilon = 2^{-m}$. Near any other number x the gap scales with x:

$$\text{gap}(x) \approx \varepsilon\,|x| \qquad (\text{between } \tfrac{1}{2}\varepsilon|x| \text{ and } \varepsilon|x|)$$

A sum $x + a$ is rounded to the nearest float, so if $|a|$ is below half the gap, $x + a$ comes out equal to x.

Worked example (different numbers, same idea)

Picture a calculator that keeps **4 significant digits**. Then $1 + 0.001 = 1.001$ is stored fine, but $1 + 0.0001 = 1.0001$ is rounded to 1.000: its epsilon is about 10^{-3}. Near $50\,000$ (stored as 5.000×10^4) the last digit is worth 10, so $50\,000 + 1$ gives $50\,000$ again. The relative gap stays about ε; the absolute gap grows with the number. (32-bit floats, common on GPUs, have $m = 23$: $\varepsilon \approx 1.19 \times 10^{-7}$.)

Overflow and underflow

- The largest double is about 1.80×10^{308}. Beyond it you get **overflow**: `1e308 * 10` gives `inf`.
- The smallest normal double is about 2.23×10^{-308}. Far below it you get **underflow** to zero: `np.exp(-800.0)` gives `0.0`.
- Large intermediate powers can overflow even when the answer fits: rescale (densities in units of 10^{20} m^{-3}) or use logarithms.

Tips and pitfalls

- **Never test floats with** `==` after arithmetic: `0.1 + 0.2 == 0.3` is `False` (the sum is `0.30000000000000004`), because 0.1 has no exact binary form. Use `abs(a - b) < tol`.
- `np.spacing(x)` gives the exact gap at x. A relative error near 10^{-16} is as exact as a double gets.